OBIEZUO EMMANUEL | e.obiezuo6230@miva.edu.ng
2026/A/MIT/0693_301926230

MIT8103 - ADVANCED DATABASE SYSTEM

HOSPITAL DATABASE ARCHITECTURE DESIGN

Case Study Selection
My database portfolio case study design is based on Hospital/clinic system, I selected GraceCare Hospital, a fictional outpatient clinic. The system entails patient registration, appointments, consultations, diagnoses, prescriptions, bills and payment. I chose this scenario because it is an area of interest for my next software development project and the system needs reliable transactions as well as efficient retrieval of medical history.

Portfolio 1: Design and modelling
I designed a relational schema in PostgreSQL and utilized primary keys, foreign keys, unique constraints and check constraints to protect data quality. The ER diagram shows how patients and doctors relate to appointments, and how consultations connect to diagnoses, prescriptions, bills and payments. The tables demonstrate strict normalization to avoid common insertion,update and deletion anomalies.

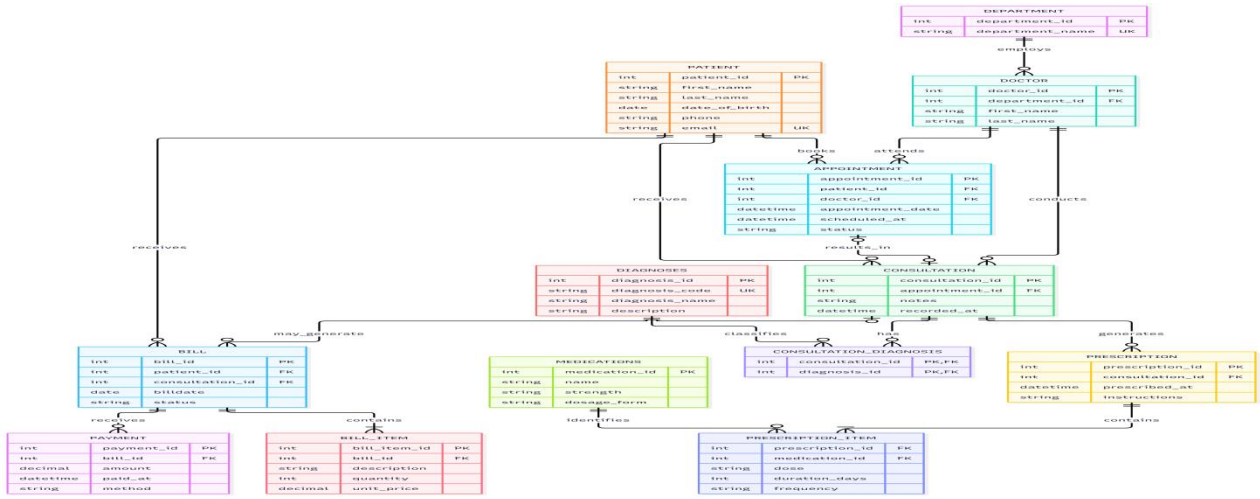


Figure 1: ER Diagram demostrating relationship between tables

Portfolio 2: Query processing and optimization
I performed queries for upcoming appointments, patient history and appointment counts. I compared query plans using `EXPLAIN (ANALYZE, BUFFERS)` before and after an index experiment.
During the experiment I witnessed a significant performance enhancement when querying tables with pre-defined indexes in attached Figure 3. Session B than querying sequentially without an index in Figure 3. Session A. Session A was excuted in 38.035 ms with 104 shared buffer hits while Session B was excuted within 6.286 ms with 10 shared buffer hits, this indicated a drastic reduction in the query time with huge margin 31.749 ms.

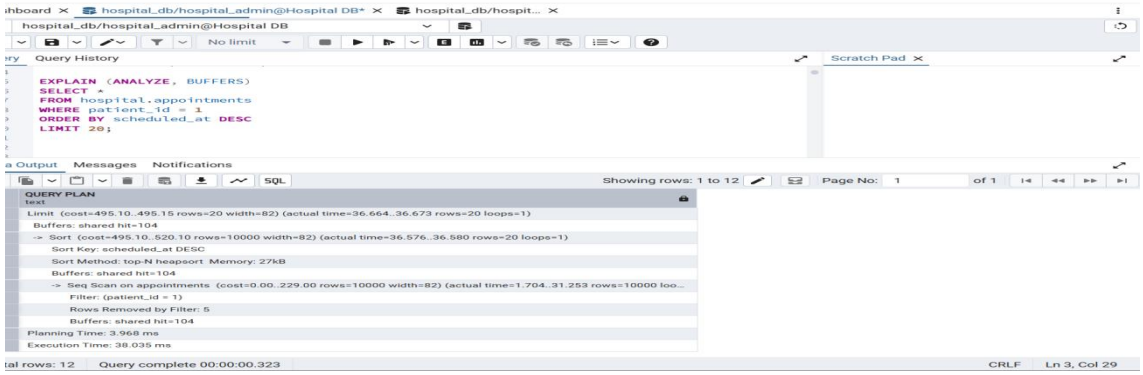


Figure 2. Session A - Query plan before index optimization

OBIEZUO EMMANUEL | e.obiezuo6230@miva.edu.ng
2026/A/MIT/0693_301926230

MIT8103 - ADVANCED DATABASE SYSTEM

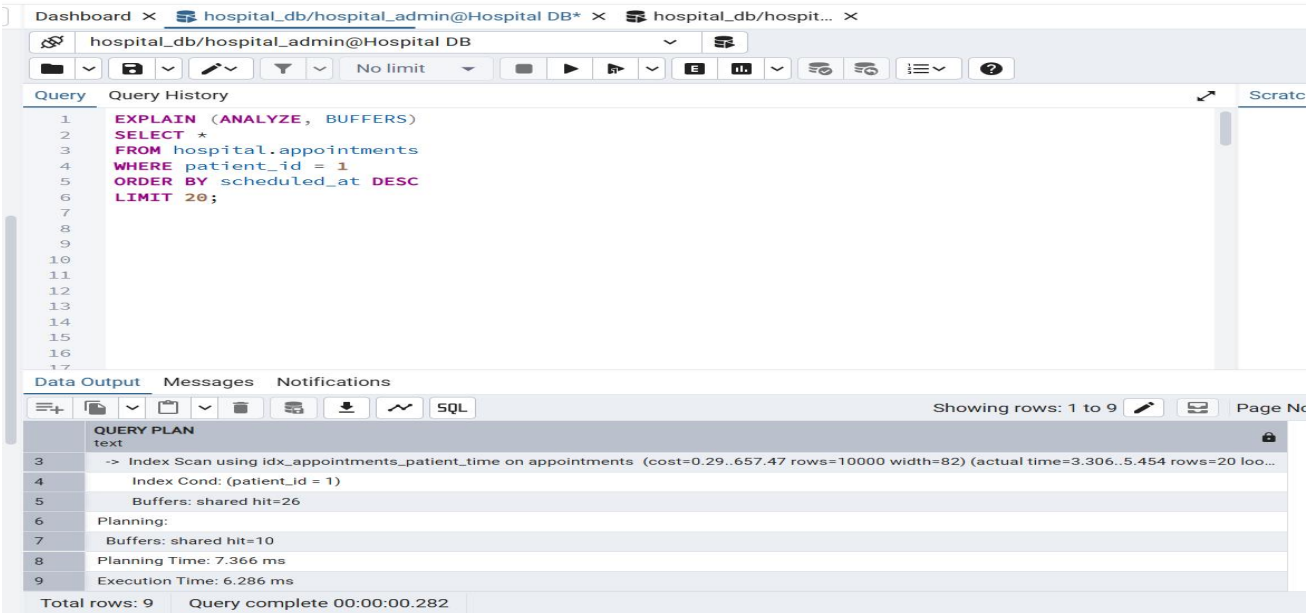


Figure 3: Session B - showing Query plan after index optimization

Portfolio 3: Transactions and concurrency
I tested `COMMIT` and `ROLLBACK` and examined how a unique constraint prevents duplicate appointments for the same doctor and time. I also explored transaction isolation using two sessions .

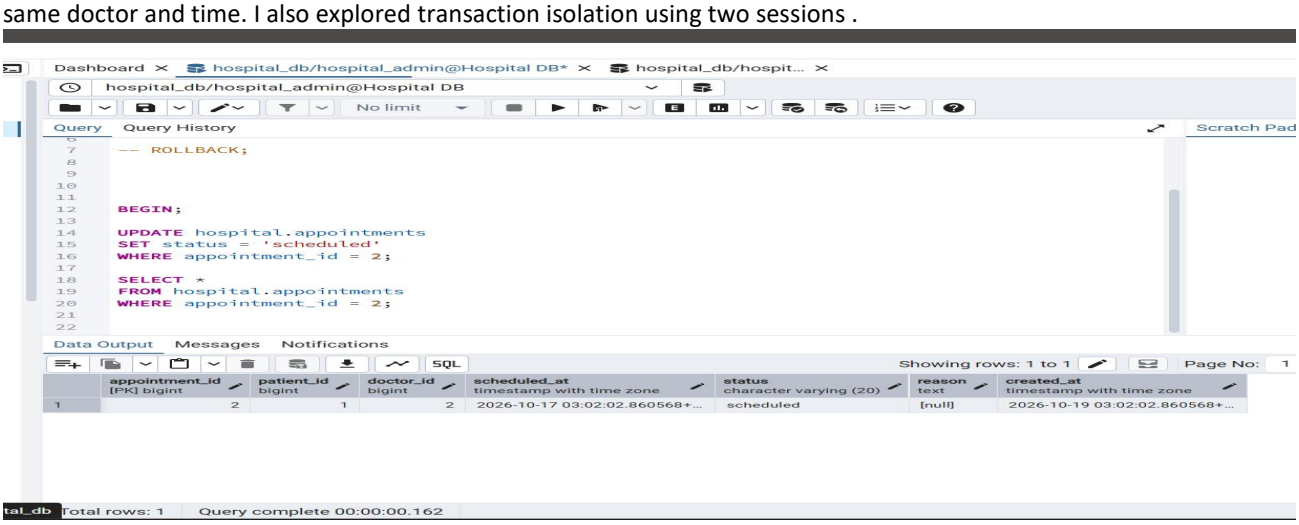


Figure 4: concurrency transaction-Section A

OBIEZUO EMMANUEL | e.obiezuo6230@miva.edu.ng
2026/A/MIT/0693_301926230

MIT8103 - ADVANCED DATABASE SYSTEM

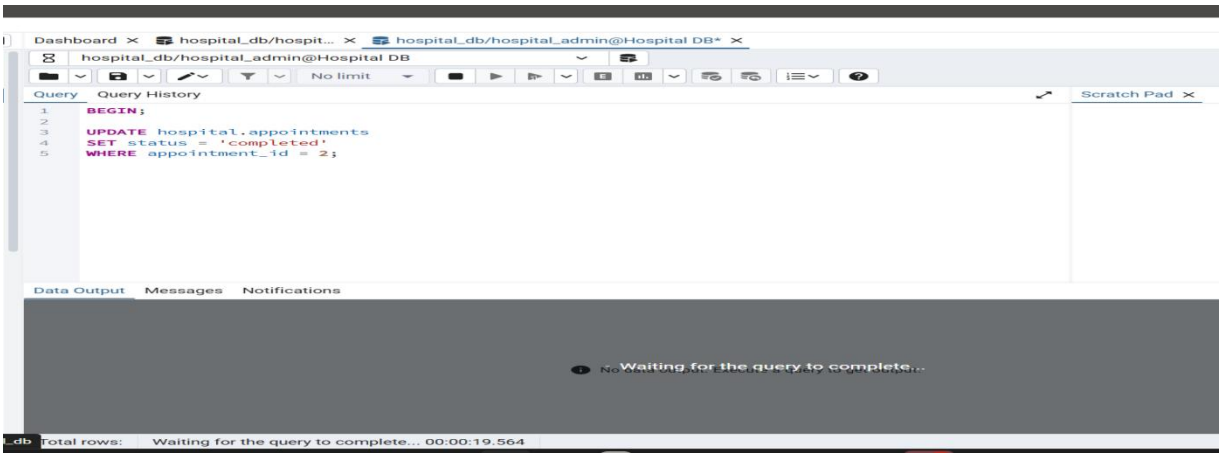


Figure 5: Concurrency transaction-Section B

Concurrency test procedure:
Session A starts a transaction (BEGIN;) to update the status of appointment_id = 2 to 'scheduled'. Because the transaction was started but no COMMIT or ROLLBACK command was issued yet, the database placed a row-level lock on appointment_id = 2 While Session A was still active and holding the lock, a second tab session B attempted to start a new transaction to update the exact same appointment_id = 2 to a status of 'completed'. As shown in the screenshot, Session B was blocked and placed in a "Waiting for the query to complete..." state

Portfolio 4: NoSQL
I created a MongoDB document example for the hospital consultation summary. Embedding related information can make a summary convenient to retrieve and access all required data without multiple JOIN clauses, while PostgreSQL remains better suited to enforcing relational integrity across connected entities.

Portfolio 5: Distributed/cloud database
I examined a primary cloud database design,exploring key concepts such as replication which ensures availability of a databse by coping data on multiple serves, so that in the case of fault with one server the database will fail-over to the next server. Also examined how sharding and partitioning of a cloud database lag, fail-over, backups and consistency trade-offs.

Reflection
Before this practical, I never appreciated the ACID property of Relational database and only thought choosing database and DBMS was dependent on personal choice or skill. Now i understand concurrency and isolation in transactions, it use cases in critical systems like the hospital or banking . Building the portfolio helped me see that the design also affects data quality, query performance and what happens when several users act on the same records. One challenge I encountered was having to perform JOIN method on sql tables to have a comprehensive data return unlike nosql documents. I addressed it by performing more queries with the syntax. If I extended the project, I would improve on my query performance strategy and test it with a larger fictional dataset.

Repository and declarations
GitHub repository: https://github.com/emmyVon/advanced_database_portfolio

AI-use declaration
AI tool used: ChatGPT,Google Gemini.
Purpose: to help plan the portfolio structure, clarify database concepts and generate fictional dataset.
My verification: I reviewed the generated material, executed and tested the scripts that I submit, corrected issues found during testing, captured my own evidence and wrote the reflection to describe my actual work.